

```
In [1]: 1 import pandas as pd
        2 import numpy as np
        3 import matplotlib.pyplot as plt
        4 from sklearn.tree import DecisionTreeClassifier
        5 from sklearn.model_selection import train_test_split
        6 from sklearn.metrics import accuracy_score
```

Import the data

```
In [2]: 1 df = pd.read_csv('D:\dynamic language\heart.csv')
        2 # Display the first few rows of the DataFrame
        3 df
```

Out[2]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	284	1	1	106	0	1.9	1	3	2	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1020	59	1	1	140	221	0	1	164	1	0.0	2	0	2	1
1021	60	1	0	125	258	0	0	141	1	2.8	1	1	3	0
1022	47	1	0	110	275	0	0	118	1	1.0	1	1	2	0
1023	50	0	0	110	254	0	0	159	0	0.0	2	0	2	1
1024	54	1	0	120	188	0	1	113	0	1.4	1	1	3	0

1025 rows × 14 columns

```
In [3]: 1 # Data Dimensions
        2 df.shape
```

Out[3]: (1025, 14)

```
In [4]: 1 # Values of each row
        2 df.values
```

Out[4]: array([[52., 1., 0., ..., 2., 3., 0.],
 [53., 1., 0., ..., 0., 3., 0.],
 [70., 1., 0., ..., 0., 3., 0.],
 ...,
 [47., 1., 0., ..., 1., 2., 0.],
 [50., 0., 0., ..., 0., 2., 1.],
 [54., 1., 0., ..., 1., 3., 0.]])

clean data

In [5]:

```
1 df.describe()
2 #no missing
```

Out[5]:

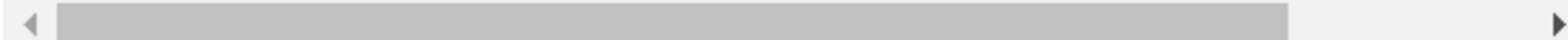
	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1024.000000	1025.000000	1025.000000
mean	54.434146	0.695610	0.942439	131.611707	246.000000	0.149268	0.529756	149.114146	0.336585	1.072266	1.385366	0.750000
std	9.072290	0.460373	1.029641	17.516718	51.59251	0.356527	0.527878	23.005724	0.472772	1.175380	0.617755	1.000000
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000	0.000000	0.000000	0.000000
25%	48.000000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000	132.000000	0.000000	0.000000	1.000000	0.000000
50%	56.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.000000	152.000000	0.000000	0.800000	1.000000	0.000000
75%	61.000000	1.000000	2.000000	140.000000	275.000000	0.000000	1.000000	166.000000	1.000000	1.800000	2.000000	1.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000	202.000000	1.000000	6.200000	2.000000	4.000000

In [6]:

```
1 # fix the missing value in col with mode
2 df["Year"].fillna(df["Year"].mode()[0], inplace=True)
3 df.describe()
```

Out[6]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000
mean	54.434146	0.695610	0.942439	131.611707	246.000000	0.149268	0.529756	149.114146	0.336585	1.071220	1.385366	0.750000
std	9.072290	0.460373	1.029641	17.516718	51.59251	0.356527	0.527878	23.005724	0.472772	1.175283	0.617755	1.000000
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000	0.000000	0.000000	0.000000
25%	48.000000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000	132.000000	0.000000	0.000000	1.000000	0.000000
50%	56.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.000000	152.000000	0.000000	0.800000	1.000000	0.000000
75%	61.000000	1.000000	2.000000	140.000000	275.000000	0.000000	1.000000	166.000000	1.000000	1.800000	2.000000	1.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000	202.000000	1.000000	6.200000	2.000000	4.000000



Split data into train set and test set

In [13]:

```
1 # Extract the target col from data
2 # Target is what the model predict
3 x = df.drop(columns = ['target'])
4 y = df["target"]
```

In [14]:

```
1 x_train , x_test , y_train , y_test = train_test_split(x,y,test_size=0.2)
```

Create model

```
In [16]: 1 # Use the decision tree algorithm
        2 model = DecisionTreeClassifier()
```

Train the model

```
In [18]: 1 # Use fit function to train model
        2 model.fit(x_train,y_train)
```

Out[18]: DecisionTreeClassifier()

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Make predications

```
In [20]: 1 pre = model.predict(x_test)
```

```
In [22]: 1 pre
```

Out[22]: array([0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0,
1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0,
0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1,
0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0,
1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0,
1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1,
1, 1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 1,
1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0,
1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1,
1, 0, 1, 0, 1, 1, 0]) dtype=int64)

Evalute and improve

```
In [21]: 1 score = accuracy_score(y_test,pre)
        2 score
```

Out[21]: 1.0

```
In [ ]: 1
```

Import mnist dataset

```
In [1]: 1 # help import mnist dataset
        2 import tensorflow as tf
        3 from tensorflow.keras import datasets, layers , models
        4 import pandas as pd
```

Split data into train set and test set

```
In [2]: 1 (train_images , train_labels ),( test_images , test_labels ) = datasets.mnist.load_data()
```

```
In [3]: 1 # normalize the image
        2 # pixel images ranges from 0 to 255
        3 # i need to make ranges from 0 to 1
        4 train_images , test_images = train_images/255 , test_images/255
```


CNN Model

```
In [4]: 1 model = models.Sequential()
2 # add feature extraction layer
3 # filter 28 with size 3*3
4 #28-3+1=26
5 model.add(layers.Conv2D(28,(3,3), activation = 'relu', input_shape = (28,28,1))) #output_image = (26,26,1)
6 model.add(layers.MaxPooling2D((2,2))) #output_image = (13,13,1)
7 model.add(layers.Conv2D(28,(3,3), activation = 'relu')) #output_image = (11,11,1)
8 model.add(layers.MaxPooling2D((2,2))) #output_image = (5,5,1)
9 model.add(layers.Conv2D(28,(3,3), activation = 'relu')) #output_image = (3,3,1)
10 model.add(layers.Flatten())
```

D:\anaconda\anaconda1\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
In [5]: 1 #gives summary about all the layers
2 model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 28)	280
max_pooling2d (MaxPooling2D)	(None, 13, 13, 28)	0
conv2d_1 (Conv2D)	(None, 11, 11, 28)	7,084
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 28)	0
conv2d_2 (Conv2D)	(None, 3, 3, 28)	7,084
flatten (Flatten)	(None, 252)	0

Total params: 14,448 (56.44 KB)

Trainable params: 14,448 (56.44 KB)

Non-trainable params: 0 (0.00 B)

Dense Layer (classification)

```
In [6]: 1 model.add(layers.Dense(64, activation = 'relu'))
2 # last layer number of neuron = number of classes
3 model.add(layers.Dense(10, activation = 'softmax'))
4 #gives array with 9 probability choose the highest probability class
```

Compiler

```
In [7]: 1 model.compile(optimizer='adam',
2               loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=['accuracy'])
```

Train the model

```
In [8]: 1 model.fit(train_images, train_labels, epochs = 5, validation_data = (test_images , test_labels))
```

Epoch 1/5

```
D:\anaconda\anaconda1\Lib\site-packages\keras\src\backend\tensorflow\nn.py:602: UserWarning: "`sparse_categorical_crossentropy` received `from_logits=True`, but the `output` argument was produced by a Softmax activation and thus does not represent logits. Was this intended?"
  output, from_logits = _get_logits(
```

```
1875/1875 ----- 12s 4ms/step - accuracy: 0.8663 - loss: 0.4256 - val_accuracy: 0.9833 - val_loss: 0.0519
Epoch 2/5
1875/1875 ----- 7s 4ms/step - accuracy: 0.9804 - loss: 0.0631 - val_accuracy: 0.9869 - val_loss: 0.0382
Epoch 3/5
1875/1875 ----- 8s 4ms/step - accuracy: 0.9863 - loss: 0.0424 - val_accuracy: 0.9893 - val_loss: 0.0313
Epoch 4/5
1875/1875 ----- 7s 4ms/step - accuracy: 0.9895 - loss: 0.0329 - val_accuracy: 0.9882 - val_loss: 0.0355
Epoch 5/5
1875/1875 ----- 7s 4ms/step - accuracy: 0.9922 - loss: 0.0262 - val_accuracy: 0.9889 - val_loss: 0.0349
```

```
Out[8]: <keras.src.callbacks.history.History at 0x1e157c3a610>
```

Make predications

```
In [9]: 1 test_digits = test_images[0:10]
        2 pre = model.predict(test_digits)
        3 pre[0]
```

1/1 ----- 0s 111ms/step

```
Out[9]: array([4.0268688e-07, 3.2422208e-06, 9.7696227e-07, 3.8919829e-06,
              7.1199693e-08, 5.8129325e-07, 1.5067735e-12, 9.9998832e-01,
              2.3468624e-06, 1.1336890e-07], dtype=float32)
```

```
In [10]: 1 pre[0].argmax()
```

```
Out[10]: 7
```

```
In [11]: 1 test_labels[0]
```

Out[11]: 7

Evaluate and improve

```
In [12]: 1 test_loss, test_accuracy = model.evaluate(test_images, test_labels)
```

```
313/313 ----- 1s 2ms/step - accuracy: 0.9857 - loss: 0.0452
```

```
In [13]: 1 print("Test Accuracy :", test_accuracy)
```

```
Test Accuracy : 0.9889000058174133
```

```
In [ ]: 1
```

```
In [ ]: 1
```